# Introduction to Computer Simulation II Project

Project by: - **Deeksha Gopal (2130194)**
Guidance: - **Dr. J. Finkenrath**

## Task-1:-

**Objective:-** The objective of this task is to implement a parallel vector product using MPI functions, specifically *MPI Reduce and MPI Bcast or MPI Allreduce.*

**Explanation:-**

- The program initializes two vectors, $\boldsymbol{v}$ and $\boldsymbol{w}$ with sample data.
- Each process computes a local sum of the vector product for its assigned elements.
- The MPI_Allreduce function is used to combine the local sums from all processes, obtaining the global sum.
- The program prints the result of the vector product for each process, demonstrating the parallel computation.

**Result:-**

```
Result of vector product on process 0: 2686600.00
Result of vector product on process 2: 2686600.00
Result of vector product on process 1: 2686600.00
Result of vector product on process 3: 2686600.00
```

**Explanation:-**

- The program initializes two vectors, $\boldsymbol{v}$ and $\boldsymbol{w}$ with sample data.
- Each process computes a local sum of the vector product for its assigned elements.
- The local sums are reduced using MPI_Reduce to obtain the global sum.
- Use MPI Reduce to collect the final result on process 0.
- The program prints the result of the vector product on process 0.

**Result:-**

```
Result of vector product on process 0: 2686600.00
```

Based on the application/ task user can use MPI_Reduce or MPI_Allreduce functions.

## Task-2:-

**Objective:-** Implementation of the Gram-Schmidt orthogonalization algorithm in a parallelized manner using MPI.

**Result:-**

```
Process 0: Orthogonal vector (q0): [0.00 0.00 0.01 0.01 0.01 0.01 0.01 0.01 0.02 0.02 0.02 0.02 0.02 0.02 0.03 0.03 0.03 0.03 0.03 0.03 0.04 0.0
4 0.04 0.04 0.04 0.04 0.05 0.05 0.05 0.05 0.05 0.06 0.06 0.06 0.06 0.06 0.06 0.07 0.07 0.07 0.07 0.07 0.07 0.08 0.08 0.08 0.08 0.08 0.08 0.09 0.
09 0.09 0.09 0.09 0.09 0.10 0.10 0.10 0.10 0.10 0.10 0.11 0.11 0.11 0.11 0.11 0.12 0.12 0.12 0.12 0.12 0.12 0.13 0.13 0.13 0.13 0.13 0.13 0.14 0
.14 0.14 0.14 0.14 0.14 0.15 0.15 0.15 0.15 0.15 0.15 0.16 0.16 0.16 0.16 0.16 0.17 0.17 0.17 0.17 0.17 ]
Process 1: Orthogonal vector (q1): [0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.0
0 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.
00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 -
0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 ]
Process 2: Orthogonal vector (q2): [0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.0
0 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.
00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 -
0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 ]
Process 3: Orthogonal vector (q3): [0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.0
0 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.
00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 0.00 -
0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 -0.00 ]
Process 0: Dot product between q0 and q1: 0.000000
Process 0: Dot product between q0 and q2: 0.000000
Process 0: Dot product between q0 and q3: 0.000000
Process 1: Dot product between q1 and q2: 0.000139
Process 1: Dot product between q1 and q3: 0.000206
Process 2: Dot product between q2 and q3: 0.000406
```

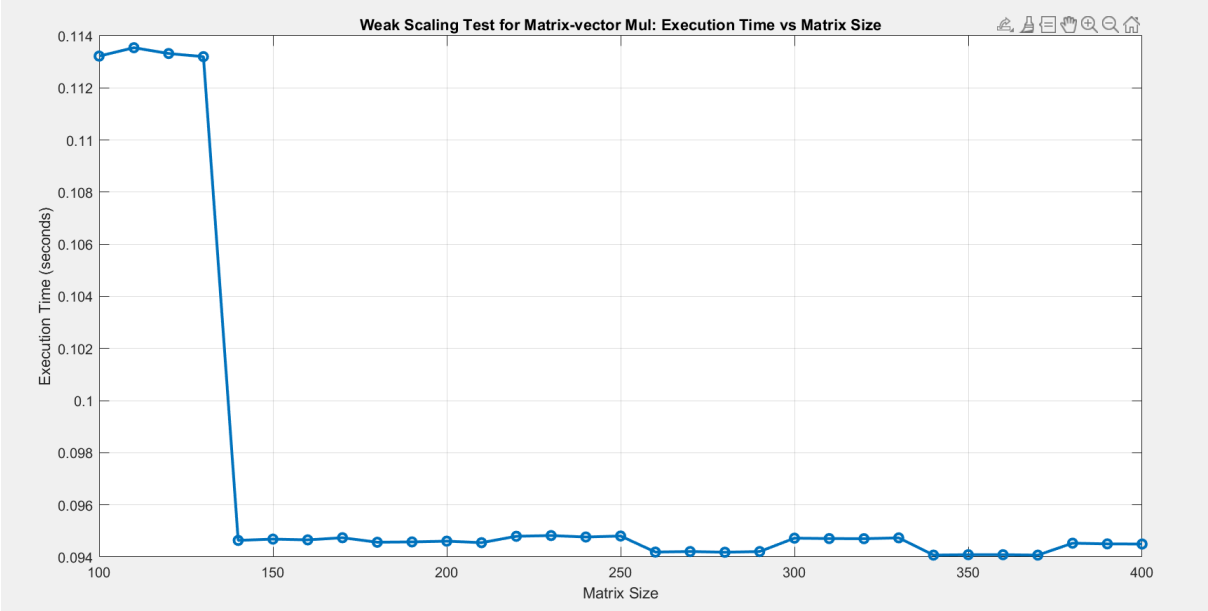
The Gram-Schmidt orthogonalization process is performed on the vectors, producing orthogonal vectors generated by each process. It even checks for the orthogonality of the vectors by computing and prints the dot product between pairs of orthogonal vectors. The dot products are utilized to verify orthogonality. The results show that the dot products between vectors of different processes are close to zero, confirming orthogonality.

## Task-3, Task-4, Task-5 & Task-7:-

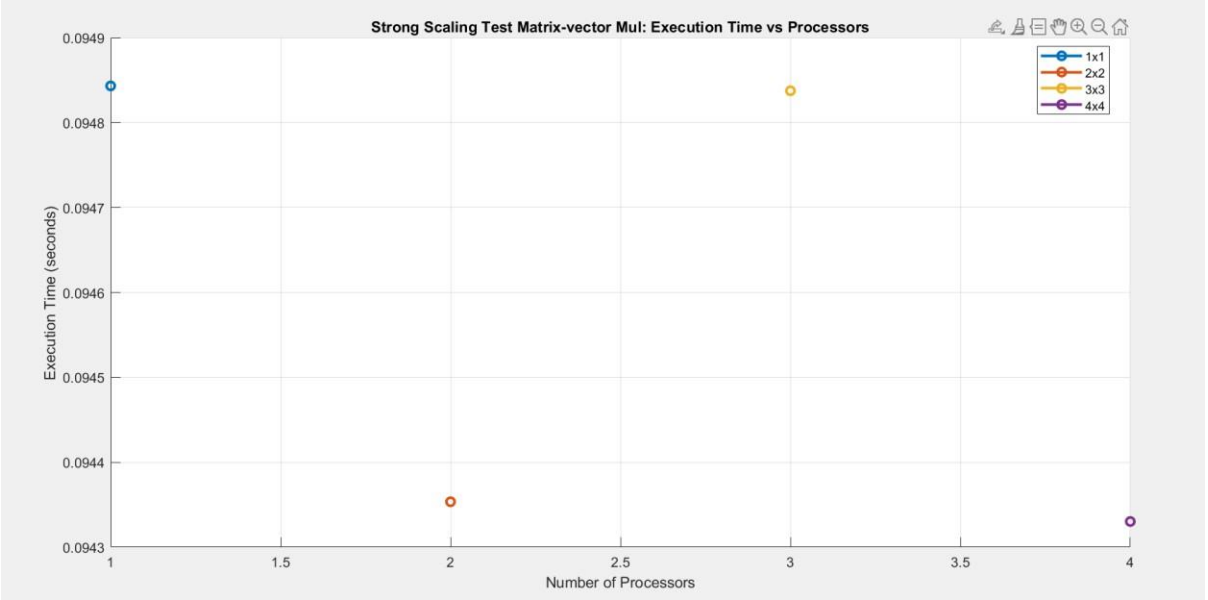
**Objective:-** Parallel matrix stencil, Jacobi iteration, steepest descent, and performing there weak and strong scaling

## Result:-

## Weak scaling of Parallel matrix-vector multiplication:-




## Strong scaling of Parallel matrix-vector multiplication:-




### Weak Scaling :-

As the matrix size increases (e.g., from 100x100 to 400x400), the execution time remains relatively stable. This is a positive outcome, indicating that the parallel algorithm can efficiently handle larger problem instances without a significant increase in execution time.

The execution times for each matrix size are consistent, suggesting that adding more processors scales well with the increased workload.

**Interpretation**:-

The parallel algorithm demonstrates good weak scaling behaviour, as it maintains a consistent execution time per processor even with larger problem sizes. This is indicative of effective load balancing and scalability in handling larger data.

**Strong Scaling:-**

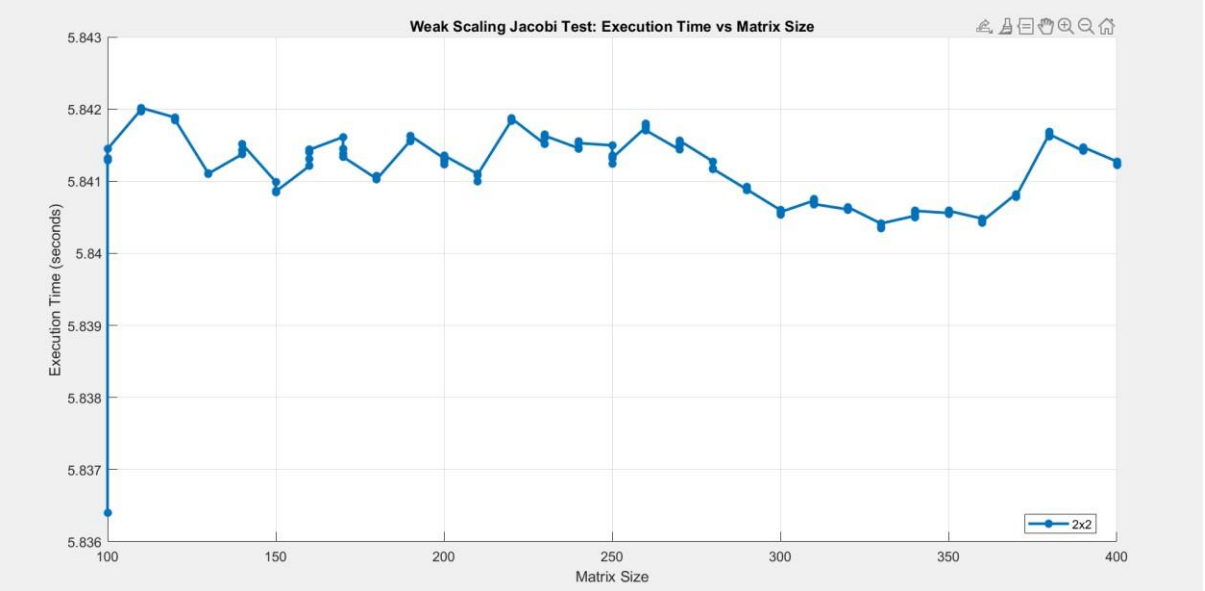
As the number of processors increases (e.g., from 1 to 4), the execution time decreases. This reduction in execution time indicates that the parallel algorithm benefits from additional processors for the given matrix size.

However, the reduction in execution time may not be perfectly linear, and there might be diminishing returns as the number of processors increases.

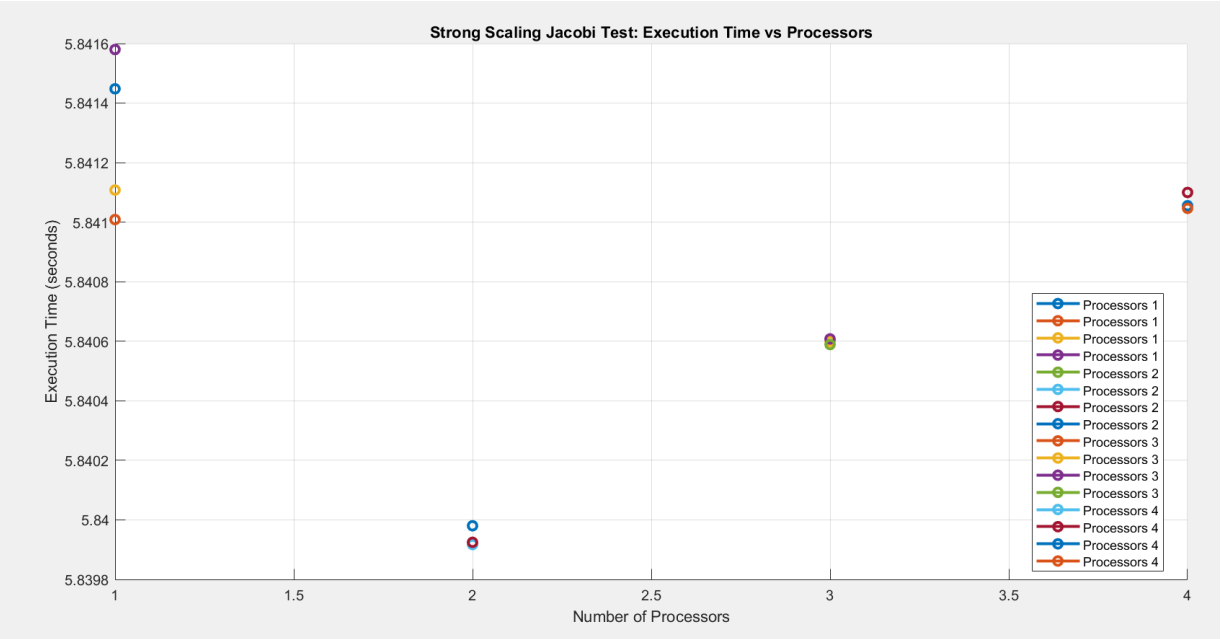
**Interpretation:-**

The parallel algorithm exhibits strong scaling behaviour, showing that increasing the number of processors leads to a reduction in execution time for a fixed-size problem. This suggests that the parallelization is effective in exploiting parallelism to solve the problem more quickly.

## Weak scaling of Jacobi Iteration:-



## Strong scaling of Jacobi Iteration:-




### Weak scaling:-

The tests are conducted on a 100x100 matrix size with 2x2 processors.

The execution time is consistently around 5.84 seconds for various matrix sizes (100x100, 110x110, 120x120, ..., 400x400), indicating that as the matrix size increases, the execution time remains relatively stable.

This suggests that the algorithm scales well with the problem size when the number of processors is increased proportionally.

## Strong Scaling:-

The tests are performed on a fixed 400x400 matrix size with varying numbers of processors (1, 2, 3, 4), with execution time is provided for each case.

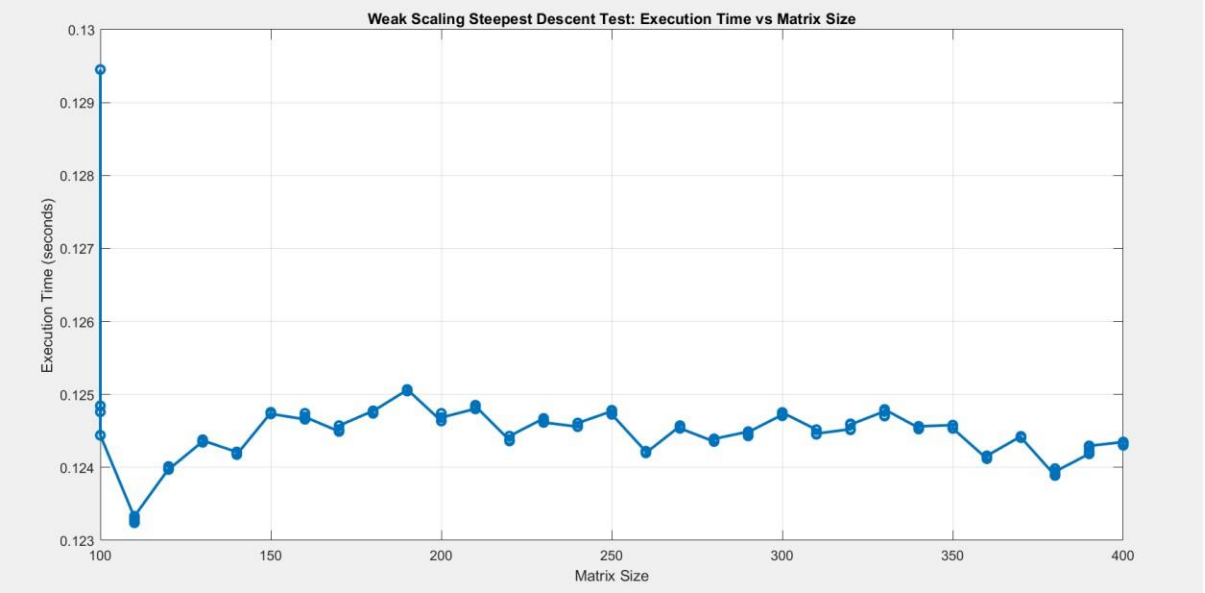
As the number of processors increases, the execution time decreases, indicating good strong scaling.

However, the speedup achieved is not perfectly linear. For example, going from 1 to 2 processors shows a reduction in execution time, but the subsequent increase in the number of processors doesn't result in a proportional decrease in execution time.

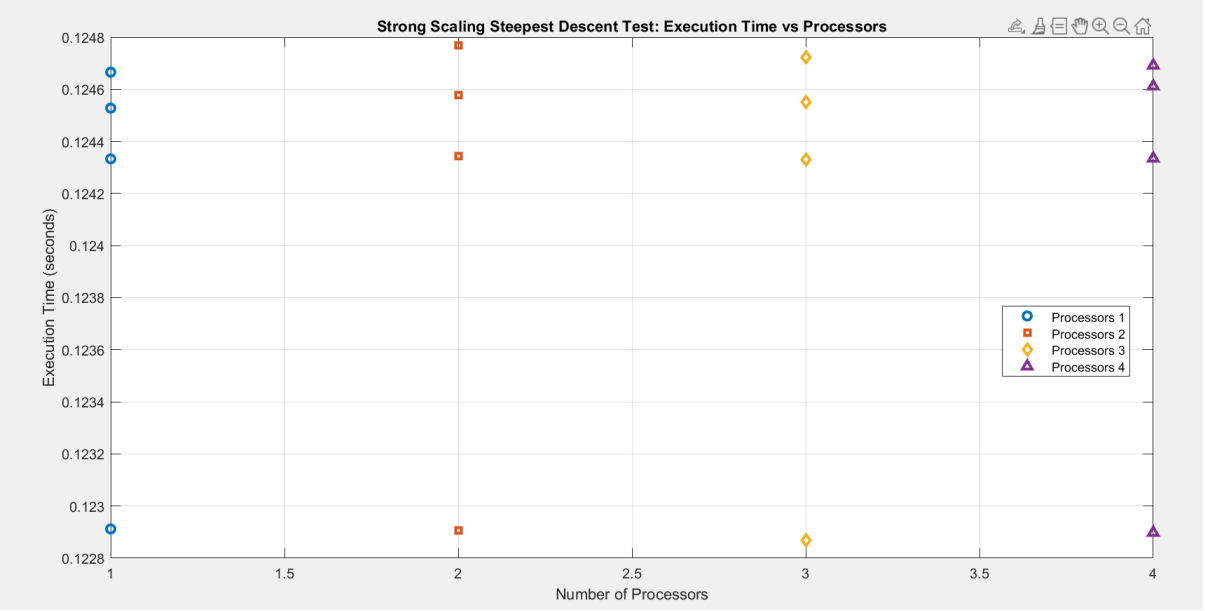
In the weak scaling scenario, the algorithm demonstrates stability in performance as the problem size grows. In the strong scaling scenario, the algorithm shows improvement in execution time with an increasing number of

processors, but the efficiency gains might diminish as the number of processors becomes larger.

## Weak scaling of Steepest descent:-




## Strong scaling of Steepest descent:-




## Weak Scaling:-

The matrix size is increasing, but the execution time remains relatively stable (around 0.124 seconds). This suggests that as you increase the problem size and the number of processors, the algorithm can handle the increased workload efficiently.

## Strong Scaling:-

The matrix size is held constant, but the execution time varies as the number of processors changes. The results indicate that as you increase the number of processors from 1 to 2 to 3 to 4, the execution time decreases (e.g., 0.124333 seconds for 1 processor to 0.122898 seconds for 4 processors). This suggests good strong scaling behaviour, adding more processors leads to a proportionate reduction in execution time.

In summary, these results suggest that the Steepest Descent algorithm shows good scalability, both in handling larger problem sizes (weak scaling) and efficiently utilizing additional processors (strong scaling).

→

$$\alpha u + ((1+\eta)\gamma(x+y)-\eta)\frac{\partial u}{\partial x} + ((1+\eta)\gamma(x-y) - \eta)\frac{\partial u}{\partial x} - \frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial y^2}$$

Discretizing spatial Derivatives with $O(h^2)$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}$$

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2}$$

$$\frac{\partial u}{\partial x} \approx \frac{u_{i+1,j} - u_{i-1,j}}{2h}$$

$$\frac{\partial u}{\partial y} \approx \frac{u_{i,j+1} - u_{i,j-1}}{2h}$$

→ substitute discretized derivatives into PDE:-

$$\alpha u_{i,j} + ((1+\eta)\gamma(x_i+y_i)-\eta)\frac{u_{i+1,j} - u_{i-1,j}}{2h}$$

$$+ ((1+\eta)\gamma(x_i-y_i)-\eta)\frac{u_{i,j+1} - u_{i,j-1}}{2h}$$

$$- \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2} - \frac{u_{i,j+1} - 2u_{i,j} + u_{i,j-1}}{h^2}$$

$$= b_{i,j}$$

arrange terms to represent stencil:-

$$\alpha u_{i,j} + \frac{(1+\eta)\gamma(x_i+y_i)-\eta}{2h}(u_{i+1,j} - u_{i-1,j}) + \frac{(1+\eta)\gamma(x_i-y_i)-\eta}{2h}(u_{i,j+1} - u_{i,j-1}) - \frac{1}{h^2}(u_{i+1,j} - 2u_{i,j} + u_{i-1,j}) - \frac{1}{h^2}(u_{i,j+1} - 2u_{i,j} + u_{i,j-1}) = b_{i,j}$$

Normalize the diagonal:-

$$u_{i,j} + \frac{\left(\frac{(1+\eta)\gamma(x_i+y_i)-\eta}{2h}\right)}{\alpha}(u_{i+1,j} - u_{i-1,j}) + \frac{\left(\frac{(1+\eta)\gamma(x_i-y_i)-\eta}{2h}\right)}{\alpha}(u_{i,j+1} - u_{i,j-1}) - \frac{\frac{1}{h^2}}{\alpha}(u_{i+1,j} - 2u_{i,j} + u_{i-1,j}) - \frac{\frac{1}{h^2}}{\alpha}(u_{i,j+1} - 2u_{i,j} + u_{i,j-1}) = \frac{b_{i,j}}{\alpha}$$

$$u_{i,j} + \frac{(1+\eta)\gamma(x_i+y_i)-\eta}{2\alpha h}u_{i+1,j} + \frac{(1+\eta)\gamma(x_i-y_i)-\eta}{2\alpha h}u_{i,j+1} - \left(\frac{(1+\eta)\gamma(x_i+y_i)-\eta}{2\alpha h}u_{i-1,j} + \frac{(1+\eta)\gamma(x_i-y_i)-\eta}{2\alpha h}u_{i,j-1}\right) - \frac{1}{\alpha h^2}u_{i+1,j} - \frac{1}{\alpha h^2}u_{i-1,j} - \frac{1}{\alpha h^2}u_{i,j+1} - \frac{1}{\alpha h^2}u_{i,j-1} + \frac{2}{\alpha h^2}u_{i,j} = \frac{b_{i,j}}{\alpha}$$

$$A_{i,j} = 1 + \frac{2\alpha h(1+\eta)\gamma(x_i+y_i) - \eta + 2\alpha h(1+\eta)\gamma(x_i-y_i) - \eta + \alpha h^2}{u_{i,j}}$$

$$E_{i,j} = -\frac{2\alpha h(1+\eta)\gamma(x_i+y_i) - \eta}{u_{i+1,j}} - \frac{\alpha h^2}{u_{i+1,j}}$$

$$F_{i,j} = -\frac{2\alpha h(1+\eta)\gamma(x_i-y_i) - \eta}{u_{i-1,j}} - \frac{\alpha h^2}{u_{i-1,j}}$$